Government Data APIs

Week 11 · APIs module · Textbook Chapter 11

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

October 26, 28 & 30, 2026

This week at a glance

Four public APIs, each showing one idea that holds for every API. This week, every class is live coding, from one lab notebook in three parts.

Monday | Lab, part 1: the Census

- Keys, and *what can I ask for?* — the variable and geography catalogs

Wednesday | Lab, part 2: FRED, then the EIA

- *Let the server compute*, then *let the server filter*

Friday | Lab, part 3: Congress.gov

- *Follow the links*; join two APIs; then textbook revisions

The lab

`labs/week-11/lab-11.ipynb` in the course repository. Each part is added before its day.

Four keys, as environment variables

`CENSUS_KEY` today; `FRED_KEY` and `EIA_KEY` Wednesday; `DATAGOV_KEY` Friday. Links are in the Canvas announcement and the lab's first cell.

Due Sunday, Nov 1

Chapter 11's **Recommended Exercises**, from its companion notebook. Graded on good-faith attempt to complete rather than getting everything working.

1. Monday · Live-Coding Lab

This week, every class is live coding

Find it | `labs/` in the course repository

1. Open the **labs** folder of `cuinfoscience/INF04617-Fall2026` on GitHub
2. Open `week-11`, click `lab-11.ipynb`, then **Download raw file**
3. Open it in JupyterLab in the `webdata` environment, *after* setting your keys

Use it | one part a day

- I write the code on screen, and you write it in your copy
- Put **your** email in the User-Agent in Step 0
- Today: Part 1, Steps 0–6, about 50 minutes
- Wednesday and Friday: download the new copy and start at that part's setup step

Solutions

`lab-11-key.ipynb`, beside it: every cell filled in and run. Fall behind? Copy the cell from the key and keep going.

Nothing to submit

The lab is practice. After each step, a **Discuss** box holds questions we'll talk over before the next.

Error flavors

A missing key stops Step 0 *on purpose*: set it and run again. A cell that fails with your key set is *broken*: tell me, or open an issue.

No key yet?

Follow along: the key shows every output. A Census key works only after you click the link in its email.

Public data by mandate

Government APIs exist because of democratic commitments to **transparency** — not business strategy:

- The **Census Bureau** is constitutionally mandated to count the population.
- The **Federal Reserve** publishes the economic data it collects; FRED is the St. Louis Fed's service for it.
- The **Energy Information Administration** was created in 1977, after the energy crises, to publish independent energy statistics.
- The **Library of Congress** publishes Congress's legislative record at Congress.gov.

They embody the **openness** value from Ch. 3, *The Post-API Age*: public data, freely accessible by mandate rather than platform goodwill.

Stable, but less polished

Agencies don't pivot business models or get acquired — so these APIs are unusually **stable**. The tradeoff: dense docs, cryptic variable names, unhelpful errors. Navigating that *is* the skill.

Four APIs, four ideas

Week 10's Wikipedia taught **pagination**. Each API this week teaches one more idea you'll meet in every API:

Day	API	The idea
Monday	Census Bureau	**Discovery**: ask the API what you can ask for
Wednesday	FRED	**Let the server compute**: transforms in the request
Wednesday	EIA	**Let the server filter**: facets narrow the data before it's sent
Friday	Congress.gov	**Follow the links**: records point to related records

And one idea across all four: a **key** says who you are, and it stays out of your notebook.

Keys stay out of the notebook

A **key** identifies you, counts your requests, and enforces rate limits. A key typed into a notebook travels with it: into a commit, a submission, a classmate's laptop.

Store it with the `webdata` environment, once, in a terminal (on Windows, the Anaconda Prompt); then restart Jupyter and read it:

```
conda env config vars set CENSUS_KEY="your-key" -n webdata
```

```
import os
census_key = os.environ["CENSUS_KEY"] # a missing key stops here
```

The User-Agent still says who sent each request: `WebDataScience/1.0 (your-email@colorado.edu)`.

Or in your shell

`export CENSUS_KEY="your-key"` in `~/.zshrc` or `~/.bashrc`; `setx` on Windows. Chapter 11's "Setting Environment Variables" shows both ways.

Or a file

A JSON file, `api_keys.json`, added to `.gitignore` *before your first commit*. A key in git history stays there even after you delete the file.

Names

This course uses `CENSUS_KEY`, `FRED_KEY`, `EIA_KEY`, and `DATAGOV_KEY`. Chapter 11 uses the same names.

Discovery: two catalogs before any data

What | the variable catalog

- Variables come in **groups**: `B25034` is "Year Structure Built"
- `B25034_002E`: group, line, and `E` for estimate (`M` is its margin of error)
- `.../acs5/groups/B25034.json` is itself an API

Where | the geography catalog

- `.../acs5/geography.json` lists every level the dataset answers for
- Each says what it **requires**: a `place` needs a `state`; a `tract` needs a `state` and a `county`
- Places are **FIPS codes**, as strings: Colorado `08`, Boulder `07850`

Codes change between survey years: learn to look them up, not to memorize them. Then **sanity-check** one value: Boulder's median household income is $87,493 (±$6,221), not 7 or 70 million.

Census — from the catalogs to a real place

Name the variables with `get`, the place with `for`, and what it requires with `in`:

```
census_base = "https://api.census.gov/data/2024/acs/acs5"
params = {"get": "NAME,B25034_001E,B25034_002E,B25034_003E",
          "for": "place:07850", # Boulder: a FIPS code, kept as a string
          "in": "state:08", # a place requires its state
          "key": census_key}
data = requests.get(census_base, params=params, headers=HEADERS).json()

df = pd.DataFrame(data[1:], columns=data[0]) # row 0 is the header
numeric_cols = [c for c in df.columns if c.startswith("B25034")]
df[numeric_cols] = df[numeric_cols].apply(pd.to_numeric) # values arrive as strings
```

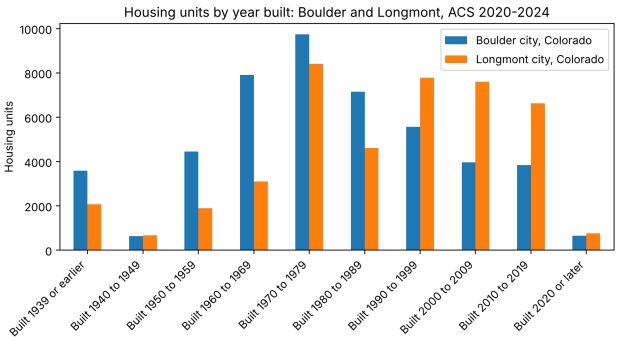
Boulder has 47,465 housing units; 646 were built in 2020 or later, 3,836 in the 2010s.

Two cities, ten decades

Step 4 asks for all ten decades of `B25034`, labels them with Step 1's catalog, and puts the oldest at the left:

```
housing_vars = [f"B25034_{line:03d}E"
                for line in range(2, 12)]
```

Boulder's biggest decade was the 1970s; Longmont has more units from every decade since the 1990s. The chart counts what still stands in 2020–2024.



Drawn by the chapter's code from the Census Bureau's API, 2026-10-25 (chapter 11, figure 11.5).

When there's no estimate: `-666666666`

83 of Colorado's 482 places have no median-income estimate in 2020–2024: too few households answered. The API doesn't leave the value out — it sends a number that can't be an income.

Place	People	Median income
Boulder	106,433	87,493
Campo	58	53,125 (±40,478)
Bonanza	20	`-666666666`
Carbonate	0	`-666666666`

Average it as is

The mean of that column is −333,298,178. Mask the codes first: `towns.mask(towns < 0)`.

What the codes mean

The Census Bureau's notes on estimate and annotation values list each one; a margin of error of `-222222222` goes with it.

2. Wednesday · Live-Coding Lab

Today: Part 2, FRED and then the EIA

Both of today's APIs do work *before* they answer:

- **FRED** (Federal Reserve Economic Data, from the St. Louis Fed) **computes**: changes, percent changes, yearly averages
- The **EIA** (U.S. Energy Information Administration) **filters**: one state, one sector, one column, one span of years

Asking the server, instead of downloading everything and doing it in pandas, is today's idea.

Use it | Steps 7–14, about 50 minutes

- Download the new `lab-11.ipynb`; start at **Step 7**, which imports and reads the keys again
- Put **your** email in the User-Agent

Two keys today

`FRED_KEY` and `EIA_KEY`, set *before* you start Jupyter. Both APIs call the parameter `api_key`; Monday's Census called it `key`.

No key yet?

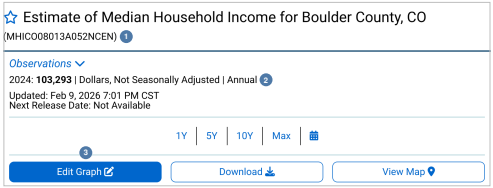
The EIA's arrives by email within minutes. Follow along from the key until it does.

FRED: find the series, then ask what it is

Every one of FRED's 800,000 series has an **ID** (1), on its page. The `series` endpoint describes it; `series/observations` sends its values:

```
fred_base = "https://api.stlouisfed.org/fred"
params = {"series_id": "MHICO08013A052NCEN",
          "api_key": fred_key, "file_type": "json",
          "observation_end": "2024-12-31"}
data = requests.get(f"{fred_base}/series/observations",
                    params=params, headers=HEADERS).json()
income = pd.DataFrame(data["observations"])[["date", "value"]]
income["value"] = pd.to_numeric(income["value"],
                                errors="coerce") # "." is missing
```

2024: $103,293 (2) — not the Census API's $103,994. Different programs: name the one you cite.



Boulder County's median household income on FRED, captured 2026-10-25 (chapter 11, figure 11.2).

Let the server compute: `units`

Step 9 gathers Step 8's lines into `fred_observations(series_id, **params)`. Then the *same series* twice more, one parameter changed:

```
change = fred_observations("MHICO08013A052NCEN",
           units="chg", observation_end="2024-12-31")
growth = fred_observations("MHICO08013A052NCEN",
           units="pch", observation_end="2024-12-31")
```

Year	Dollars	Change	Percent change
2022	96,584	6,257	6.9
2023	97,170	586	0.6
2024	103,293	6,123	6.3

What `units` can be

`lin`	as published
`chg`	change
`ch1`	change from a year before
`pch`	percent change
`pc1`	percent change from a year before
`log`	natural logarithm

Microsprint

In which years did the income *fall*? Six, from 2001 to 2020; the largest, $3,003 in 2009.

Let the server compute: `frequency`, then `units`

One series, `CPIAUCNS` (the national Consumer Price Index, monthly), three requests:

```
years = {"observation_start": "2015-01-01",
         "observation_end": "2025-12-31"}
monthly = fred_observations("CPIAUCNS", **years)
annual = fred_observations("CPIAUCNS", frequency="a",
           aggregation_method="avg", **years)
inflation = fred_observations("CPIAUCNS", frequency="a",
           aggregation_method="avg", units="pc1", **years)
```

FRED lowers the frequency first, then applies the units: each year's mean of twelve months, then its percent change from the year before.

Trust, but check once

Microsprint: the same numbers from `monthly`, with `groupby()` and `pct_change()`.

Year	FRED	pandas
2023	4.1	4.1
2024	2.9	2.9
2025	2.6	2.6

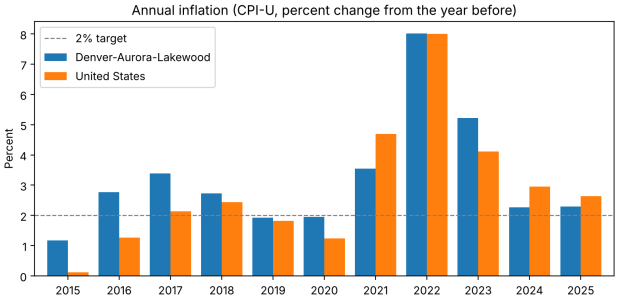
Why can FRED give 2015's inflation, and pandas can't?

Denver against the nation

The Bureau of Labor Statistics' CPI for the Denver area, `CUUSA433SA0`, is annual already: it needs only `units="pc1"`.

```
denver = fred_observations(
    "CUUSA433SA0", units="pc1", **years)
cpi = denver.merge(inflation, on="date",
    suffixes=("_denver", "_national"))
```

Denver ran ahead from 2015 to 2020, and behind since 2024. Housing is the largest part of the CPI: what would you ask FRED next?



Drawn by the chapter's code from FRED, 2026-10-25 (chapter 11, figure 11.6). The dashed line is the Federal Reserve's 2% target.

EIA: routes that describe themselves

The EIA's API is a tree of **routes**. Ask for one without `data/`, and it says what it holds:

```
eia_base = "https://api.eia.gov/v2"
meta = requests.get(f"{eia_base}/electricity/retail-sales/",
                    params={"api_key": eia_key},
                    headers=HEADERS).json()["response"]
print([f["id"] for f in meta["facets"]])
# ['stateid', 'sectorid']
for column, about in meta["data"].items():
    print(column, "|", about["units"])
# price | cents per kilowatt-hour ...and 3 more
```

A **facet** filters the rows; a **data column** (1) is what each row holds. Monday's idea, *what can I ask for?*, from a different API.



The EIA's API Dashboard, captured 2026-10-25 (chapter 11, figure 11.3): its menus (2) are those routes, and it writes the request (3).

Let the server filter

Add `data/` to the route, and each parameter narrows the rows. The brackets are part of the names:

```
params = {"frequency": "annual", "data[]": "price",
          "facets[stateid][]": "CO",
          "facets[sectorid][]": "RES", # homes
          "start": "2001", "end": "2025",
          "sort[0][column]": "period",
          "sort[0][direction]": "asc",
          "api_key": eia_key}
prices = requests.get(f"{eia_base}/electricity/retail-sales/data/",
            params=params, headers=HEADERS).json()["response"]
print("Rows:", prices["total"]) # Rows: 25
```

25 rows, from a route that holds every state and sector by month, quarter, and year. Colorado's homes paid 15.69 cents a kilowatt-hour in 2025.

Microsprint

Change one facet: a sector (`COM`, `IND`) or a state (`UT`, `NM`). Colorado's businesses paid 12.37 cents in 2025.

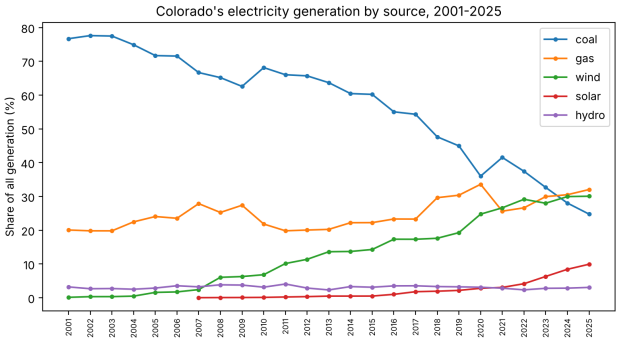
Still strings

Prices and `total` arrive as text, like the Census Bureau's numbers: `pd.to_numeric()` before any math.

What a filtered request can show

The chapter asks the same API for Colorado's annual generation from five sources, plus the total: one facet, six values, one request.

Coal made 76.7% of the state's electricity in 2001 and 24.7% in 2025. Wind grew from almost nothing to 30.0%; 2024 was the first year coal wasn't the largest source.



Drawn by the chapter's code from the EIA's API, 2026-10-25 (chapter 11, figure 11.7).

Paging: `offset` and `length`, against Wikipedia's `continue`

EIA | you keep count

- At most 5,000 rows a request; `total` says how many match
- Colorado's monthly generation, 2001–2025: 9,289 rows
- Page two: `offset=5000`, the rows to skip
- Stop when you have `total` rows — the `"incomplete return"` warning comes with the last page too

Wikipedia | the server keeps count

- Week 10: each answer carried a `continue` token
- You sent it back without knowing what it meant
- Stop when there's no token

Sort when you page, so no row moves between pages. And **filter first**: a facet, a shorter span, or a lower frequency may fit the answer in one page.

3. Friday · Live-Coding Lab

Today: Part 3, Congress.gov and a join

Lab | Steps 15–20, about 35 minutes

- **Congress.gov**, from the Library of Congress: a bill's record is mostly **links** to other records. Today's idea: *follow the links*.
- Then a **join**: the Census Bureau's incomes with the EIA's electricity bills, state by state
- Download the new `lab-11.ipynb`; start at **Step 15**

Then | the textbook, 15 minutes

- What the lab turned up in chapter 11, as a pull request

Today's key goes in a header

`DATAGOV_KEY`, sent as `X-Api-Key`, which every api.data.gov service accepts. Congress.gov's links can then be sent as they arrive, and no URL you print holds the key.

Also today

`CENSUS_KEY` and `EIA_KEY`, for the join.

A bill is mostly links

```
congress_headers = HEADERS.copy()
congress_headers["X-Api-Key"] = datagov_key

bill = requests.get(f"{congress_base}/bill/119/hr/7730",
          params={"format": "json"},
          headers=congress_headers).json()["bill"]
for field, value in bill.items():
    if isinstance(value, dict) and "url" in value:
        print(field, value.get("count"), value["url"])
# actions 18 .../bill/119/hr/7730/actions?format=json
# cosponsors 6 .../bill/119/hr/7730/cosponsors?format=json
# textVersions 5 .../bill/119/hr/7730/text?format=json
```

Each link is a complete request: send it with the same headers. The code never builds an address.



H.R. 7730 (1), its sponsor (2), and its tracker (3), on Congress.gov, October 2026 (chapter 11, figure 11.4).

Follow the links

```
actions = requests.get(bill["actions"]["url"],
             params={"limit": 250},
             headers=congress_headers).json()["actions"]

milestones = ("Introduced in", "Passed/agreed to in",
             "Became Public Law")
shown = set()
for action in reversed(actions): # newest first
    if (action["text"].startswith(milestones)
            and action["text"] not in shown):
        print(action["actionDate"], action["text"][:40])
        shown.add(action["text"])
# 2026-02-26 Introduced in House
# 2026-09-16 Passed/agreed to in House: ...
# 2026-09-28 Passed/agreed to in Senate: ...
```

The cosponsors' link

Six cosponsors. One of the three original ones is Joe Neguse, Boulder's representative (CO-2).

Recorded twice

Some milestones appear twice, in the same words: print each once.

Microsprint

Follow `textVersions`: how many versions of the text, and what is the newest called?

Paging with `next`: the week's third way

API	The response gives you	You send	Stop when
Wikipedia (week 10)	a `continue` token	the token, back	there's no token
EIA (Wednesday)	`total`	`offset`, which you count	you have `total` rows
Congress.gov (today)	`pagination.next`, a whole URL	the URL, as it is	there's no `next`

Step 18 takes Joe Neguse's sponsored legislation in three moves: two pages by hand, one page as a function you write, `bills_on_page()`, then the given loop, `sponsored_bills()`.

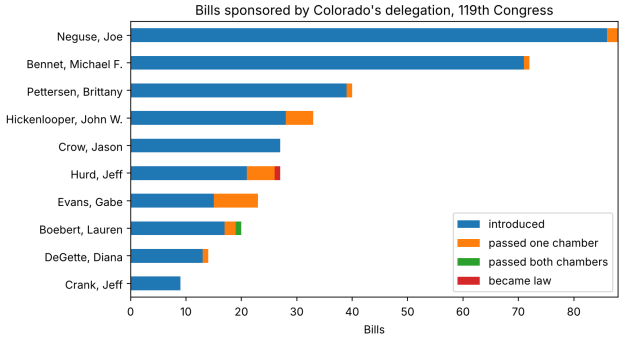
Microsprint: another member of Colorado's delegation. How many of their bills are still in committee?

Following every link: how far the bills got

The chapter follows the `actions` link of every bill Colorado's ten members sponsored in the 119th Congress: **353 requests.**

326 were only introduced; 25 passed one chamber; 1 passed both; 1 became law (H.R. 2316).

A count of milestones isn't a measure of effectiveness: Volden and Wiseman weigh each bill by what it would change. And the chapter's `fromDateTime` asks only for what changed since the last run.



Drawn by the chapter's code from Congress.gov's API, as of October 25, 2026 (chapter 11, figure 11.8).

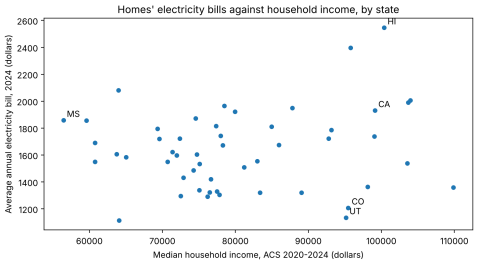
Two agencies, one identifier

The Census Bureau calls Colorado `08`; the EIA calls it `CO`. A **crosswalk** pairs the codes:

```
states = states.merge(codes[["STATEFP", "STATE"]],
           left_on="state", right_on="STATEFP", how="left")
joined = states.merge(power, left_on="STATE",
           right_on="stateid", how="outer", indicator=True)
```

Look at what didn't match: Puerto Rico, on the Census side; ten regions and the U.S. total, on the EIA's. An inner join drops them without a word.

Bills as a share of income: Mississippi and Alabama 3.3%; Colorado 1.3%, $1,207 a year.



Drawn by the chapter's code from the Census Bureau's and the EIA's APIs, 2026-10-25 (chapter 11, figure 11.9).

Friday: from the lab to the textbook (15 minutes)

Look back at what the lab turned up in chapter 11, and propose one change as a **pull request**:

- **An output that has changed.** The chapter printed its outputs on October 25, while Congress was in session. Compare H.R. 7730's latest action, or a member's number of bills.
- **A cell that broke.** A Congress.gov link sent without its key fails with a 403. Does "Common Issues to Debug" say so clearly enough?
- **An entry to test.** "More Government APIs for Colorado" lists twelve more. Try one: is its entry right?
- **A microsprint that would make a good exercise.** The text versions, or a member's bills still in committee.

How to propose one

The course's `handouts/common/revision-framework.md` says what each kind of change needs to be accepted.

A good review

Run the change; name the line; separate "must fix" from "nice to have"; end with a verdict.

Key takeaways

1. Government APIs serve **public mandates**: the census, the Federal Reserve's data, the EIA's statistics, Congress's record.
2. Each carried one idea you'll meet again: **ask what you can ask for** (Census), **let the server compute** (FRED), **let the server filter** (EIA), **follow the links** (Congress.gov).
3. **Keys stay out of notebooks**: environment variables, and a header where the API allows one.
4. **Three ways to page**: a token, an offset, a next link. Know which one you're using, and when it stops.
5. **A join is only as good as its identifiers**: use a crosswalk, and look at the rows that didn't match.